

Frontend Engineering Challenge: Dynamic Asset Configuration

Overview

Welcome! In this take-home assignment, you will build a dynamic configuration form for an electrical asset management system. Our platform manages different types of equipment, and the metadata required varies significantly depending on the `assetType`.

The Task

Your task is to build a React component that renders a dynamic form based on a predefined configuration schema. You will use our core technology stack: **React Hook Form**, **Zod**, and **Material UI (MUI)**.

Requirements

1. Dynamic Rendering:

Create a single form component that accepts an `assetType` prop (e.g., "TRANSFORMER", "SECTION", "BREAKER").

- If the type is "TRANSFORMER", render a text input for `kvaRating` (must be a positive number) and a select dropdown for `coolingType` (options: ONAN, ONAF).
- If the type is "SECTION", render a boolean switch/checkbox for `groundedNeutral` and a text input for `conductorType` (minimum 3 characters).

2. Schema Validation (Zod):

- Use **Zod** to dynamically define validation rules based on the active `assetType`.
- Integrate Zod with React Hook Form to handle client-side validation.
- Show error messages clearly using MUI form control error states.

3. Strict TypeScript:

- Define strict TypeScript interfaces for your form payloads.
- **Do not use any types.** Ensure the `onSubmit` payload is perfectly typed according to the `assetType`.

4. Testing (Vitest):

- Write tests using Vitest and React Testing Library.
- Follow a Test-Driven approach: verify that invalid inputs show the correct validation errors, and valid inputs trigger the `onSubmit` callback with the correctly structured payload.

Setup Instructions

If you are setting this up locally, you can use Vite or Create React App. Ensure you install:

```
npm install react-hook-form @hookform/resolvers zod @mui/material @emotion/react @emotion/styled
npm install -D vitest @testing-library/react @testing-library/jest-dom
```

Deliverables

- Source code for the form components and Zod schemas.
- Test files demonstrating validation logic.
- Instructions on how to run your component and tests.

Good luck! We look forward to seeing your clean, type-safe implementation.